

Test Plan — JBS IntelliQE (End-to-End Application Coverage)

Scope. This plan validates the **JBS IntelliQE platform itself** — the multi-tenant AI QA-automation product — not the customer applications it generates tests for. It covers the complete workflow from authentication, through the Chat wizard's seven-source requirement intake, the six-agent AI pipeline (requirements → planning → generation → scripting → execution → healing → reporting), git publishing, and every admin/management surface, plus cross-cutting concerns (multi-tenancy, security, convergence guards).

Authoring standard. Every case follows `docs/test-case-authoring-standards.instructions.md` : TestRail *Test Case (Steps)* schema with **Module / Sub-Module / Section Hierarchy**, a dedicated **Test Data** column (simulated/generated values tagged), and an **Expected Result for every step**. IDs `TC-001...` are stable across regenerations.

1. Plan Metadata

Field	Value
Product	JBS IntelliQE (TestOps-IntelliQE)
Version under test	Frontend VITE_APP_VERSION build; Backend Express API :3001; Worker process
Plan version	1.0
Last updated	2026-08-10
Total test cases	84
Test types	Functional, Negative, Edge, E2E, Smoke, Security, Accessibility, Regression

1.1 Modules covered

#	Module	Cases
A	Authentication & Session	TC-001 – TC-010
B	Chat Wizard — Requirement Sources	TC-011 – TC-024
C	Chat Wizard — Generation & Review	TC-025 – TC-034
D	Chat Wizard — Scripts, Execution, Healing	TC-035 – TC-046
E	Chat Wizard — Reporting & Git Publish	TC-047 – TC-052
F	Generated Test Cases	TC-053 – TC-060
G	Reports	TC-061 – TC-063
H	Bug Tracker	TC-064 – TC-070
I	System Configuration	TC-071 – TC-078
J	User Management & Feature Toggles	TC-079 – TC-082
K	Cross-cutting — Multi-tenancy, Security, Pipeline Guards	TC-083 – TC-084

1.2 Test environment & prerequisites

- Backend API running on `:3001` ; frontend dev/prod build proxying `/api` ; at least one **worker** process running (`npm run worker:start`) for the DB-queued pipeline paths.
- Azure SQL / SQL Server reachable, schema `JBSTestOpsAI` initialized.
- A **platform tenant** (`jbs` , `is_platform=true`) and at least one **customer tenant**.
- Seed users per role: `admin` , `qa_engineer` , `data_analyst` .
- A valid **Anthropic API key** configured (per-tenant LLM config) for any AI-pipeline case.
- A public **application-under-test (AUT)** with a login (e.g. the OrangeHRM demo) configured under **System Configuration** → **Application Setup** for generation/execution cases.
- Sandbox credentials for integrations: Jira, Azure DevOps, Confluence, SharePoint, GitHub/GitLab/Bitbucket, SMTP.

1.3 Test Data conventions

- Real credentials are **never** written in this plan — use placeholders like `<valid_password>` , `<jira_api_token>` , `<anthropic_key>` .
- Values the tester must create are tagged (created by Tester) ; forced error conditions are tagged (simulated) .
- Transit-encrypted fields (`__ENC__`...) and at-rest fields (`__AES__`...) must never appear in plaintext in DevTools, logs, or API responses.

2. Module A — Authentication & Session

TC-001: Verify login page renders all required controls on load

- **Title Description:** Confirms the unauthenticated login screen presents the sign-in form and mode toggle so a user can begin authentication.
- **Module:** Authentication
- **Sub-Module:** Login Page
- **Section Hierarchy:** Authentication > Login Page
- **Priority:** Critical
- **Type:** Smoke
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** No active session (sessionStorage cleared).
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Navigate to <code>/</code> while logged out.	The app redirects to <code>/login</code> and the login card is displayed.
2	Observe the sign-in form.	Username field, Password field with a show/hide eye toggle, and a disabled "Sign In" button are displayed.
3	Observe the bottom of the card.	A toggle to switch to "Sign Up" mode is displayed.
4	Type any value into Username and Password.	The "Sign In" button becomes enabled.

TC-002: Verify successful login with valid credentials lands on Chat

- **Title Description:** Confirms a registered active user can authenticate and is routed to the main Chat wizard with a persisted session.
- **Module:** Authentication
- **Sub-Module:** Login Page
- **Section Hierarchy:** Authentication > Login Page
- **Priority:** Critical
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** An active user exists in a tenant.
- **Test Data:** username = qa_engineer_user password =
- **References:** —

Step #	Step Description	Expected Result
1	Navigate to <code>/login</code> .	The sign-in form is displayed.
2	Enter the valid username.	The Username field shows the value.
3	Enter the valid password.	The Password field shows masked characters.
4	Click "Sign In".	A "Signing in..." spinner is shown, then the user is redirected to <code>/chat</code> .
5	Inspect <code>sessionStorage</code> .	<code>intelliqe_token</code> (JWT) and <code>intelliqe_user</code> are present; the header shows the username and role.

TC-003: Verify login fails with a generic message for invalid credentials

- **Title Description:** Confirms wrong credentials are rejected with a non-revealing error and no session is created.
- **Module:** Authentication
- **Sub-Module:** Login Page
- **Section Hierarchy:** Authentication > Login Page
- **Priority:** Critical
- **Type:** Negative
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** The login page is reachable.
- **Test Data:** username = qa_engineer_user password = wrong_password_99 (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Enter a valid username and an incorrect password.	Both fields show their values with the password masked.
2	Click "Sign In".	The request returns 401 and an "Invalid username or password." message is displayed.
3	Observe the URL and storage.	The user remains on <code>/login</code> ; no token is written to sessionStorage.

TC-004: Verify login distinguishes server/network failure from bad credentials

- **Title Description:** Confirms the UI surfaces a distinct error (not "invalid credentials") when the backend is unreachable or returns 500.
- **Module:** Authentication
- **Sub-Module:** Login Page
- **Section Hierarchy:** Authentication > Login Page
- **Priority:** High
- **Type:** Negative
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Ability to stop the backend or force a 500.
- **Test Data:** backend = stopped / returns HTTP 500 (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Stop the backend (or intercept <code>/api/auth/login</code> to return 500).	The API is unreachable.
2	Enter valid credentials and click "Sign In".	An error banner shows a server/network message with a hint, distinct from the "Invalid username or password." text.
3	Observe the page.	The user remains on <code>/login</code> and can retry once the backend recovers.

TC-005: Verify password is transit-encrypted on the wire

- **Title Description:** Confirms the frontend XOR-encrypts the password (`__ENC__` prefix) before POSTing so it never appears in plaintext in DevTools.
- **Module:** Authentication
- **Sub-Module:** Transit Security
- **Section Hierarchy:** Authentication > Transit Security
- **Priority:** High
- **Type:** Security
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Browser DevTools Network tab open.
- **Test Data:** username = qa_engineer_user password =
- **References:** —

Step #	Step Description	Expected Result
1	Open DevTools → Network and submit the login form with valid credentials.	The <code>POST /api/auth/login</code> request is captured.
2	Inspect the request payload.	The <code>password</code> value carries the <code>__ENC__</code> prefix and is not the plaintext password.
3	Inspect the response.	The response contains a JWT and user object; no plaintext password is echoed.

TC-006: Verify sign-up creates an account with role selection and returns to sign-in

- **Title Description:** Confirms a new user can self-register with a chosen role and is guided back to sign-in.
- **Module:** Authentication
- **Sub-Module:** Sign Up
- **Section Hierarchy:** Authentication > Sign Up
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** The chosen username does not already exist.
- **Test Data:** full_name = New Tester (created by Tester) email = new.test@example.com (created by Tester) username = new_tester_20260810 (created by Tester) role = QA Engineer password =
- **References:** —

Step #	Step Description	Expected Result
1	On the login card, switch to "Sign Up".	Full Name, Email, Username, Role radio cards, Password and Confirm Password fields are displayed.
2	Fill all fields and select the "QA Engineer" role.	The "Sign Up" button becomes enabled.
3	Ensure Password and Confirm Password match and are ≥6 chars.	No validation error is shown.
4	Click "Sign Up".	An "Account created successfully!" message is displayed.

Step #	Step Description	Expected Result
5	Wait ~1.5 seconds.	The card auto-switches back to Sign In mode.

TC-007: Verify sign-up validation blocks mismatched or short passwords

- **Title Description:** Confirms client-side validation prevents submission when passwords differ or are under the minimum length.
- **Module:** Authentication
- **Sub-Module:** Sign Up
- **Section Hierarchy:** Authentication > Sign Up
- **Priority:** Medium
- **Type:** Negative
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** Sign Up mode is active.
- **Test Data:** password = abc12 (simulated: 5 chars) confirm_password = abc123 (simulated: mismatch)
- **References:** —

Step #	Step Description	Expected Result
1	Enter a 5-character password.	On submit, a "minimum 6 characters" validation message is displayed and the account is not created.
2	Enter a 6+ character password and a different Confirm Password.	A "passwords must match" validation message is displayed and the account is not created.
3	Correct both fields to match and meet the length.	The validation messages clear and the submit is allowed.

TC-008: Verify an inactive account cannot log in

- **Title Description:** Confirms a deactivated user is blocked at authentication even with correct credentials.
- **Module:** Authentication
- **Sub-Module:** Login Page
- **Section Hierarchy:** Authentication > Login Page
- **Priority:** High
- **Type:** Negative
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** An existing user has been set to inactive via User Management.
- **Test Data:** username = deactivated_user password = is_active = false (created by Tester)
- **References:** —

Step #	Step Description	Expected Result
1	Attempt to sign in with the deactivated user's valid credentials.	Authentication is rejected and no token is issued.
2	Observe the message and URL.	An error is displayed and the user remains on <code>/login</code> .

TC-009: Verify logout clears the session and protects routes

- **Title Description:** Confirms signing out removes session storage and prevents access to protected pages without re-authentication.
- **Module:** Authentication
- **Sub-Module:** Session
- **Section Hierarchy:** Authentication > Session
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A user is logged in.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Click the user avatar → "Sign out" in the header.	The session is cleared and the user is redirected to <code>/login</code> .
2	Inspect <code>sessionStorage</code> .	<code>intelliqe_token</code> and <code>intelliqe_user</code> are removed.
3	Manually navigate to <code>/reports</code> .	The route guard redirects to <code>/login</code> .

TC-010: Verify unauthenticated deep-link is redirected to login

- **Title Description:** Confirms any protected route accessed without a session redirects to the login page rather than rendering.
- **Module:** Authentication
- **Sub-Module:** Session
- **Section Hierarchy:** Authentication > Session
- **Priority:** Medium
- **Type:** Negative
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** No active session.
- **Test Data:** `deep_link` = `/system-configuration`
- **References:** —

Step #	Step Description	Expected Result
1	With <code>sessionStorage</code> cleared, navigate directly to <code>/system-configuration</code> .	The <code>ProtectedRoute</code> guard redirects to <code>/login</code> .
2	Navigate directly to an unknown path such as <code>/nope</code> .	The wildcard route redirects to <code>/</code> .

3. Module B — Chat Wizard: Requirement Sources

TC-011: Verify Chat welcome shows only permitted automation categories per role

- **Title Description:** Confirms Tessa greets the user and offers Web Application Automation (active) and API Automation (Coming Soon), gated by role.
- **Module:** Chat Wizard
- **Sub-Module:** Welcome / Category Select
- **Section Hierarchy:** Chat Wizard > Welcome / Category Select
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A `qa_engineer` user is logged in on a fresh Chat session.
- **Test Data:** `user` = `qa_engineer_user`
- **References:** —

Step #	Step Description	Expected Result
1	Open <code>/chat</code> in a new session.	Tessa greets the user by name and a welcome message is displayed.
2	Observe the category cards.	"Web Application Automation" is active; "API Automation" shows a "Coming Soon" badge and is disabled.
3	Click the "API Automation" card.	A "coming soon" message is shown and the flow does not advance.
4	Log in instead as a <code>data_analyst</code> and open <code>/chat</code> .	No automation categories are shown (empty welcome).

TC-012: Verify all seven requirement-source options are presented

- **Title Description:** Confirms selecting Web Application Automation reveals the seven requirement sources.
- **Module:** Chat Wizard
- **Sub-Module:** Requirement Source Select
- **Section Hierarchy:** Chat Wizard > Requirement Source Select
- **Priority:** High
- **Type:** Smoke
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** On the Chat welcome step.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Select "Web Application Automation".	The requirement-source step is displayed.
2	Observe the source cards.	JIRA, Azure DevOps, Confluence, SharePoint, Upload Document, Paste Requirements, and Explore App are all displayed.

TC-013: Verify selecting an unconfigured connector routes the user to setup

- **Title Description:** Confirms a connector source that has no saved integration shows a setup message instead of failing.
- **Module:** Chat Wizard
- **Sub-Module:** Requirement Source Select
- **Section Hierarchy:** Chat Wizard > Requirement Source Select
- **Priority:** High
- **Type:** Negative
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** No Jira integration is connected for the tenant.
- **Test Data:** `jira_connected = false` (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	On the source step, click "JIRA" with no Jira connection saved.	A message states JIRA "isn't configured yet. Please set up the connection under System Configuration first."
2	Observe the flow.	The wizard stays on the source-select step; no story fetch is attempted.

TC-014: Verify Jira story selection fetches details and advances to column select

- **Title Description:** Confirms a connected Jira source lists stories and, on selection, pulls full story details for generation.
- **Module:** Chat Wizard
- **Sub-Module:** JIRA Stories
- **Section Hierarchy:** Chat Wizard > JIRA Stories
- **Priority:** Critical
- **Type:** E2E
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** Jira is connected (Requirement Sources) and at least one application is configured (Application Setup).
- **Test Data:** jira_connected = true application_configured = true story_key = QA-101 (created by Tester)
- **References:** QA-101

Step #	Step Description	Expected Result
1	Select "JIRA" as the source.	A "Connected to JIRA" badge appears and the story count is shown.
2	Open the story dropdown.	Stories are listed as KEY — summary .
3	Select a story and click "Proceed".	Full story details (title, description, acceptance criteria) are fetched.
4	Observe the next step.	The wizard advances to the Column Select step.

TC-015: Verify Jira flow warns when no application is configured

- **Title Description:** Confirms the wizard blocks generation grounding when Jira is connected but no AUT exists, pointing the user to Application Setup.
- **Module:** Chat Wizard
- **Sub-Module:** JIRA Stories
- **Section Hierarchy:** Chat Wizard > JIRA Stories
- **Priority:** High
- **Type:** Negative
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** Jira connected; **no** application configured (readyApps empty).
- **Test Data:** jira_connected = true application_configured = false (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Select "JIRA" with no application configured.	An amber warning about missing application configuration is displayed.
2	Pick a story and attempt to proceed.	A message directs the user to configure an application in Application Setup; the flow does not proceed to generation.

TC-016: Verify Azure DevOps "Generate from Stories" mode lists open work items

- **Title Description:** Confirms ADO connection offers a generate-from-stories path that excludes closed items.
- **Module:** Chat Wizard
- **Sub-Module:** Azure DevOps Mode
- **Section Hierarchy:** Chat Wizard > Azure DevOps Mode
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** Azure DevOps is connected.
- **Test Data:** ado_connected = true
- **References:** —

Step #	Step Description	Expected Result
1	Select "Azure DevOps".	An "Azure DevOps connected" badge and two option cards (Generate from Stories / Import Existing Test Cases) are displayed.
2	Choose "Generate from Stories".	Open/non-closed work items are fetched and the content-select dropdown is populated.
3	Observe closed work items.	Items in Done/Closed/Removed state are not listed.

TC-017: Verify Azure DevOps "Import Existing Test Cases" maps steps into results

- **Title Description:** Confirms importing ADO Test Case work items brings their steps directly into the results table and marks the requirements/design stages complete.
- **Module:** Chat Wizard
- **Sub-Module:** Azure DevOps Mode
- **Section Hierarchy:** Chat Wizard > Azure DevOps Mode
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** ADO connected with at least one Test Case work item containing steps.
- **Test Data:** ado_connected = true ado_testcase_id = 5001 (created by Tester)
- **References:** —

Step #	Step Description	Expected Result
1	Choose "Import Existing Test Cases".	ADO Test Case work items are fetched with their step tables parsed.
2	Proceed with the import.	The results step is shown with the imported cases and their steps.
3	Observe the pipeline sidebar.	Requirement Analysis and Test Design stages are marked complete.

TC-018: Verify Confluence and SharePoint sources populate the content list

- **Title Description:** Confirms connected Confluence pages and SharePoint documents each populate the generic content-select dropdown.
- **Module:** Chat Wizard
- **Sub-Module:** Content Select
- **Section Hierarchy:** Chat Wizard > Content Select
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Confluence and SharePoint are each connected.
- **Test Data:** confluence_connected = true sharepoint_connected = true confluence_space_key = QA (created by Tester)
- **References:** —

Step #	Step Description	Expected Result
1	Select "Confluence".	Recently-modified pages are listed in the content dropdown (label adapts to "pages").
2	Select a page and proceed.	The page's plain-text content is retrieved and the wizard advances to Column Select.
3	Restart and select "SharePoint".	Recent documents are listed; selecting a PDF/DOCX extracts its text and advances to Column Select.

TC-019: Verify document upload accepts supported types and enforces limits

- **Title Description:** Confirms the upload step accepts PDF/DOCX/TXT/MD up to 15 MB and requires enough extractable text.
- **Module:** Chat Wizard
- **Sub-Module:** Upload Document
- **Section Hierarchy:** Chat Wizard > Upload Document
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** On the upload-doc step.
- **Test Data:** valid_file = requirements.pdf (≥20 chars text) (created by Tester) oversized_file = big.pdf (> 15 MB) (simulated) empty_file = blank.pdf (no extractable text) (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Drag-and-drop a valid PDF with readable text.	The file is accepted; character/page counts are displayed and the wizard advances to Column Select.
2	Attempt to upload a file larger than 15 MB.	The upload is rejected with a size (413/too-large) error message.
3	Upload a document with no extractable text.	An error banner reports insufficient extractable text (≥20 chars required) and the flow does not advance.

TC-020: Verify Paste Requirements requires non-empty text

- **Title Description:** Confirms free-text requirements can be submitted and the submit control is disabled when empty.
- **Module:** Chat Wizard
- **Sub-Module:** Paste Requirements
- **Section Hierarchy:** Chat Wizard > Paste Requirements
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** On the paste-text step.
- **Test Data:** requirements_text = "As a user I can reset my password from the login page." (created by Tester)
- **References:** —

Step #	Step Description	Expected Result
1	Observe the "Submit Requirements" button with an empty textarea.	The button is disabled.
2	Type requirement text into the textarea.	The button becomes enabled.
3	Click "Submit Requirements".	The wizard advances to Column Select carrying the pasted text.

TC-021: Verify Explore App requires URL and guidance before starting

- **Title Description:** Confirms the Explore source gates the "Start Exploration" action on a required URL and a required guidance prompt.
- **Module:** Chat Wizard
- **Sub-Module:** Explore App
- **Section Hierarchy:** Chat Wizard > Explore App
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** On the explore-form step.
- **Test Data:** application_url = <https://opensource-demo.orangehrmlive.com> (created by Tester) guidance = "Focus on login and PIM add-employee flows." (created by Tester, ≤1000 chars)
- **References:** —

Step #	Step Description	Expected Result
1	Leave URL and guidance empty.	The "Start Exploration" button is disabled.
2	Enter a valid Application URL only.	The button remains disabled until guidance is provided.
3	Enter guidance text and observe the character counter.	The counter updates and stays within the 1000-char cap; the button becomes enabled.
4	Click "Start Exploration".	The wizard stores an <code>__EXPLORE__:<url></code> marker and advances to Column Select.

TC-022: Verify Explore App accepts optional login credentials for gated apps

- **Title Description:** Confirms optional username/password can be supplied so the explorer can crawl behind a login.
- **Module:** Chat Wizard
- **Sub-Module:** Explore App
- **Section Hierarchy:** Chat Wizard > Explore App
- **Priority:** Medium
- **Type:** Edge
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** On the explore-form step; a login-gated AUT.
- **Test Data:** application_url = (created by Tester) username = password = guidance = "Explore the authenticated dashboard." (created by Tester)
- **References:** —

Step #	Step Description	Expected Result
1	Enter the URL, guidance, and optional username/password (password masked, show/hide works).	All fields accept input; the password is masked by default.
2	Start exploration.	The crawl proceeds and login is attempted with the supplied credentials.
3	Inspect network/logs.	The supplied password is not echoed in plaintext in any response or log.

TC-023: Verify pre-populated application dropdown fills name and URL in Explore

- **Title Description:** Confirms choosing a configured application in the Explore form auto-fills its name and base URL.
- **Module:** Chat Wizard
- **Sub-Module:** Explore App
- **Section Hierarchy:** Chat Wizard > Explore App
- **Priority:** Low
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** At least one application configured in Application Setup.
- **Test Data:** configured_app = OrangeHRM Demo (created by Tester)
- **References:** —

Step #	Step Description	Expected Result
1	Open the optional Application dropdown in the Explore form.	Configured applications are listed.
2	Select a configured application.	The application name and URL fields are auto-populated from its config.

TC-024: Verify a previous Chat session is restored after navigation

- **Title Description:** Confirms an in-progress flow persists across page navigation because ChatPage stays mounted and state is saved to sessionStorage.
- **Module:** Chat Wizard
- **Sub-Module:** Session Persistence
- **Section Hierarchy:** Chat Wizard > Session Persistence
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A Chat flow has advanced past the welcome step.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Advance the wizard to, e.g., the Column Select step.	The flow state is written to sessionStorage (<code>quify_chat_<username></code>).
2	Navigate to <code>/reports</code> , then back to <code>/chat</code> .	The previous step and inputs are still present (flow not reset).
3	Reload the browser tab.	A "previous session restored" banner is shown and auto-hides after ~4 seconds; the flow resumes at the same step.

4. Module C — Chat Wizard: Generation & Review

TC-025: Verify column selection controls which fields are generated

- **Title Description:** Confirms the tester can select test-case columns and generation is blocked until at least one is chosen.
- **Module:** Chat Wizard
- **Sub-Module:** Column Select
- **Section Hierarchy:** Chat Wizard > Column Select
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A requirement source has been provided; on the Column Select step.
- **Test Data:** selected_columns = TC Number, Title, Steps, Expected, Priority, Type
- **References:** —

Step #	Step Description	Expected Result
1	Deselect all columns.	The "Generate Test Cases" button is disabled.
2	Select the default set of columns.	The button becomes enabled.
3	Click "Generate Test Cases".	The wizard advances to the Generating step and generation begins.

TC-026: Verify multi-application selection prompts when more than one app exists

- **Title Description:** Confirms that when several applications are configured, the wizard asks which app the tests target before grounding generation.
- **Module:** Chat Wizard
- **Sub-Module:** Application Guard
- **Section Hierarchy:** Chat Wizard > Application Guard
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** Two or more applications configured; a non-explore source in progress.
- **Test Data:** app_1 = OrangeHRM Demo (created by Tester) app_2 = Internal Portal (created by Tester)
- **References:** —

Step #	Step Description	Expected Result
1	Trigger generation with more than one app configured.	The App Select step is shown, listing the configured applications.
2	Select the intended application.	Generation proceeds grounded against the chosen app's base URL.

TC-027: Verify generation is blocked when no application is configured (non-explore)

- **Title Description:** Confirms a non-explore flow requires a configured application and directs the user to Application Setup when none exists.
- **Module:** Chat Wizard
- **Sub-Module:** Application Guard
- **Section Hierarchy:** Chat Wizard > Application Guard
- **Priority:** High
- **Type:** Negative
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** No application configured; a paste/upload/Jira source used.
- **Test Data:** application_configured = false (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Attempt to generate from pasted requirements with no application configured.	Generation does not start; a message points to Application Setup.
2	Configure one application, then retry.	The single application is auto-selected and generation proceeds.

TC-028: Verify successful generation renders IEEE-829 test cases

- **Title Description:** Confirms the generator returns structured test cases with per-step expected results and metadata.
- **Module:** Chat Wizard
- **Sub-Module:** Generation
- **Section Hierarchy:** Chat Wizard > Generation
- **Priority:** Critical
- **Type:** E2E
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A valid source, a configured application, and a working LLM config.
- **Test Data:** source = pasted requirements (created by Tester) llm_configured = true
- **References:** —

Step #	Step Description	Expected Result
1	Start generation.	The Requirement Analysis and Test Design stages animate in the pipeline sidebar with live timers.
2	Wait for completion.	The Results step is shown with a table of generated test cases.
3	Inspect a generated case.	It has a TC number, title, numbered steps each with an expected result, priority, and type.

TC-029: Verify generation error surfaces server message and returns to welcome

- **Title Description:** Confirms a genuine generation failure is reported clearly and the flow resets, distinct from "no cases returned".
- **Module:** Chat Wizard
- **Sub-Module:** Generation
- **Section Hierarchy:** Chat Wizard > Generation
- **Priority:** High
- **Type:** Negative
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Ability to force an LLM/backend error (e.g. invalid/absent Anthropic key).
- **Test Data:** anthropic_key = invalid (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Remove/invalidate the LLM config and start generation.	Generation fails and the server error message is surfaced.
2	Observe the flow.	The wizard returns to the welcome step (does not hang).
3	Force a valid run that yields zero parseable cases instead.	A <code>no_test_cases_generated</code> warning with parsed features/flows is shown, distinct from a hard failure.

TC-030: Verify generation request timeout is reported

- **Title Description:** Confirms a timed-out generation request produces a "timed out" message rather than a silent stall.
- **Module:** Chat Wizard
- **Sub-Module:** Generation
- **Section Hierarchy:** Chat Wizard > Generation
- **Priority:** Medium
- **Type:** Edge
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Ability to delay/intercept the generate response beyond the client timeout.
- **Test Data:** network = delayed beyond timeout (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Start generation and hold the response past the client timeout.	A "generation request timed out" message is displayed.
2	Observe the UI.	The wizard exits the generating state and allows a retry.

TC-031: Verify the results table supports inline edit, delete, and bulk delete

- **Title Description:** Confirms generated cases can be edited inline, deleted individually, and bulk-deleted before saving.
- **Module:** Chat Wizard
- **Sub-Module:** Results / Review
- **Section Hierarchy:** Chat Wizard > Results / Review
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** Generation has produced multiple cases; on the Results step.
- **Test Data:** edit_title = "Verify password reset with valid email" (created by Tester)
- **References:** —

Step #	Step Description	Expected Result
1	Click Edit on a case, change the title/steps/expected, and Save.	The card shows the edited values; Cancel would have discarded them.
2	Edit a Priority badge (P0–P3) and a Type badge.	The new priority and type are reflected on the card.
3	Select several cases and click "Delete Selected".	The selected cases are removed from the table.
4	Delete a single case via its row action.	Only that case is removed.

TC-032: Verify results pagination and select-all-on-page

- **Title Description:** Confirms the results table paginates and the select-all control scopes to the current page.
- **Module:** Chat Wizard
- **Sub-Module:** Results / Review
- **Section Hierarchy:** Chat Wizard > Results / Review
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** More than one page of generated cases exists.
- **Test Data:** page_size = 10
- **References:** —

Step #	Step Description	Expected Result
1	Set the page size to 10 and page through results.	Pagination controls move between pages of 10 cases.

Step #	Step Description	Expected Result
2	Click select-all on page 1.	Only the visible page's cases are selected.
3	Move to page 2.	The page-2 selection state is independent of page 1.

TC-033: Verify column visibility in results honors the earlier column selection

- **Title Description:** Confirms only the columns chosen at Column Select are rendered in the results table.
- **Module:** Chat Wizard
- **Sub-Module:** Results / Review
- **Section Hierarchy:** Chat Wizard > Results / Review
- **Priority:** Low
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** Generation started with a restricted column set (e.g. no Preconditions).
- **Test Data:** selected_columns = TC Number, Title, Steps, Expected
- **References:** —

Step #	Step Description	Expected Result
1	Generate with only TC Number, Title, Steps, Expected selected.	The results table shows those columns.
2	Observe unselected columns (Priority/Type/Preconditions).	Columns that were not selected are not displayed.

TC-034: Verify starting a new chat mid-generation discards the stale flow

- **Title Description:** Confirms the monotonic flow guard drops results from a discarded run so a new chat is not clobbered by old async output.
- **Module:** Chat Wizard
- **Sub-Module:** Flow Integrity
- **Section Hierarchy:** Chat Wizard > Flow Integrity
- **Priority:** High
- **Type:** Edge
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** A generation is running.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Start a generation and, while it runs, click "New chat".	A "Discard current flow?" confirmation modal is displayed.
2	Confirm the discard.	The wizard resets to welcome and the flow id is bumped.
3	Let the original async generation resolve.	Its results are dropped and do not appear in the new flow.

5. Module D — Chat Wizard: Scripts, Execution & Healing

TC-035: Verify Save persists the generated cases and unlocks export

- **Title Description:** Confirms saving the reviewed cases writes a test run and reveals export and script-generation actions.
- **Module:** Chat Wizard
- **Sub-Module:** Save
- **Section Hierarchy:** Chat Wizard > Save
- **Priority:** Critical
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** Reviewed cases exist on the Results step.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Click "Save Test Cases".	A success card is shown and the run is persisted (test_run + test_cases).
2	Observe available actions.	Export buttons (Excel CSV / JIRA / TestRail) and a "Generate Scripts" CTA are displayed.
3	Open Generated Test Cases page.	The saved run appears in the list for the tenant.

TC-036: Verify export produces files in each supported format

- **Title Description:** Confirms saved cases export to Excel CSV, JIRA, and TestRail formats as downloadable files.
- **Module:** Chat Wizard
- **Sub-Module:** Export
- **Section Hierarchy:** Chat Wizard > Export
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** On the Saved step.
- **Test Data:** formats = Excel CSV, JIRA, TestRail
- **References:** —

Step #	Step Description	Expected Result
1	Click "Excel CSV".	A CSV file downloads containing the saved cases.
2	Click "JIRA".	A Jira-format export file downloads.
3	Click "TestRail".	A TestRail-format export file downloads.

TC-037: Verify script generation reuses pipeline scripts or regenerates only scripts

- **Title Description:** Confirms script generation reuses DOM-grounded scripts already produced by the pipeline, or regenerates only scripts (never the full pipeline), keeping IDs aligned.
- **Module:** Chat Wizard
- **Sub-Module:** Script Generation
- **Section Hierarchy:** Chat Wizard > Script Generation
- **Priority:** Critical
- **Type:** E2E
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** Cases are saved; on the Saved step.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Click "Generate Scripts".	The Script Generation step runs (Script Generation stage animates).
2	Wait for completion.	The Script Review step lists generated <code>.spec.ts</code> files, each tagged with its test-case ID, and a count badge.
3	Compare script test-case IDs to the saved cases.	The script IDs align 1:1 with the eligible (non-data) cases.

TC-038: Verify script generation failure returns to results with a message

- **Title Description:** Confirms a scripting error or empty result returns the user to the results step with an explanation, not a dead end.
- **Module:** Chat Wizard
- **Sub-Module:** Script Generation
- **Section Hierarchy:** Chat Wizard > Script Generation
- **Priority:** Medium
- **Type:** Negative
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Ability to force a scripting failure (e.g. unreachable AUT for grounding).
- **Test Data:** `grounding_crawl` = fails (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Trigger script generation under a forced failure condition.	An error/empty message is displayed.
2	Observe the flow.	The wizard returns to the results step so the user can retry.

TC-039: Verify test execution runs the suite and shows honest per-test outcomes

- **Title Description:** Confirms executing the suite runs real Playwright specs and reports passed/failed per test with durations.
- **Module:** Chat Wizard
- **Sub-Module:** Execution
- **Section Hierarchy:** Chat Wizard > Execution
- **Priority:** Critical
- **Type:** E2E
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** Scripts exist; a reachable AUT is configured.
- **Test Data:** `app_configured` = true
- **References:** —

Step #	Step Description	Expected Result
1	Click "Execute Test Suite".	A live progress bar and per-test rows (pending/running) are shown.
2	Wait for completion.	Each row resolves to passed or failed with an icon and a real duration.
3	Observe the results summary.	Total/Passed/Failed/Duration cards reflect the actual run; no pass is fabricated.

TC-040: Verify execution with no configured app marks tests not-run with a reason

- **Title Description:** Confirms that without a reachable application, tests report `not_run` with an explanation rather than a false pass or fail.
- **Module:** Chat Wizard
- **Sub-Module:** Execution
- **Section Hierarchy:** Chat Wizard > Execution
- **Priority:** High
- **Type:** Negative
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** No application base URL is configured/reachable.
- **Test Data:** `app_configured = false` (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Execute the suite with no reachable app.	Each test row is marked <code>not_run</code> with a reason (e.g. missing Base URL).
2	Observe the pipeline sidebar.	The Execution & Validation stage is marked skipped.
3	Observe the summary.	No test is reported as passed; the not-run reason is shown.

TC-041: Verify failure diagnosis translates raw errors to plain English

- **Title Description:** Confirms failed tests show a human-readable diagnosis title and a fix hint derived from the raw Playwright error.
- **Module:** Chat Wizard
- **Sub-Module:** Execution Results
- **Section Hierarchy:** Chat Wizard > Execution Results
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A run with at least one failing test.
- **Test Data:** `failure = element-not-found / assertion / login timeout` (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Open a failed test's <code>...</code> expander on the Execution Results step.	A plain-English diagnosis title and fix hint are displayed.
2	Expand the raw error.	The full raw Playwright error is available beneath the hint.

TC-042: Verify failures auto-register in the Bug Tracker (deduped)

- **Title Description:** Confirms each failing test is registered as a bug keyed by run+test case, avoiding duplicates on re-run.
- **Module:** Chat Wizard
- **Sub-Module:** Execution Results
- **Section Hierarchy:** Chat Wizard > Execution Results
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A run produces failures.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Complete an execution with failures.	Bugs are auto-created for the failing tests (type <code>failure</code>).

Step #	Step Description	Expected Result
2	Open the Bug Tracker.	The failures appear as bugs with the run/test-case linkage.
3	Re-run the same failing tests.	No duplicate bug is created for the same run+test case (upsert).

TC-043: Verify re-run failures and re-run flaky subsets execute in place

- **Title Description:** Confirms only the selected subset (failures or flaky) re-executes, and only the triggering button spins.
- **Module:** Chat Wizard
- **Sub-Module:** Execution Results
- **Section Hierarchy:** Chat Wizard > Execution Results
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** A completed run with failures and/or healed (flaky) tests.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Click "Re-run Failures".	Only failed tests re-execute; only that button shows a spinner.
2	Click "Re-run Flaky".	Only previously healed/flaky tests re-execute in place.
3	Observe the table.	The re-run rows update with fresh outcomes; unaffected rows are unchanged.

TC-044: Verify Auto-Heal is offered only when failures exist and under the attempt cap

- **Title Description:** Confirms the Auto-Heal action appears only when there are failures and fewer than two healing attempts have been used.
- **Module:** Chat Wizard
- **Sub-Module:** Auto-Healing
- **Section Hierarchy:** Chat Wizard > Auto-Healing
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A completed run with failing tests.
- **Test Data:** healing_cap = 2
- **References:** —

Step #	Step Description	Expected Result
1	Complete a run with failures.	An "Auto-Heal & Re-Execute" button is displayed alongside "Generate Report".
2	Trigger healing and let it complete without fixing everything.	An "Attempt N of 2" badge advances.
3	Reach 2 healing attempts.	The Auto-Heal option is no longer offered; the run is treated as needing human review.

TC-045: Verify healing fixes are re-executed and reclassify healed tests as flaky

- **Title Description:** Confirms the healer's fixes are proven by a real re-run, and tests that then pass are reclassified as flaky in the Bug Tracker.
- **Module:** Chat Wizard
- **Sub-Module:** Auto-Healing
- **Section Hierarchy:** Chat Wizard > Auto-Healing
- **Priority:** High
- **Type:** E2E
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** A run with a fixable failing test.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Trigger Auto-Heal on a fixable failure.	A per-test healing log shows FIXED / STILL FAILING with error and fix details.
2	Observe the re-run.	Healed scripts are re-executed and the execution table is rebuilt from real results.
3	Check the Bug Tracker for a healed-then-passing test.	It is classified as <code>flaky</code> (failed then healed), not <code>failure</code> .

TC-046: Verify healing stops when the failure signature does not change

- **Title Description:** Confirms the convergence guard aborts healing when consecutive attempts produce identical findings (no progress).
- **Module:** Chat Wizard
- **Sub-Module:** Auto-Healing
- **Section Hierarchy:** Chat Wizard > Auto-Healing
- **Priority:** Medium
- **Type:** Edge
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** A failure the healer cannot fix (identical error each pass).
- **Test Data:** failure = unfixable / identical signature (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Trigger Auto-Heal on an unfixable failure.	The healer attempts a fix and re-runs.
2	Observe subsequent passes.	When the failure signature is identical between passes, healing stops early rather than looping.
3	Observe messaging.	A "still failing / needs human review" outcome is communicated.

6. Module E — Chat Wizard: Reporting & Git Publish

TC-047: Verify the report summarizes pass rate, healing, and not-run tests

- **Title Description:** Confirms the generated report presents a color-coded pass rate, metrics grid, and a not-run callout to prevent a "0/0 healthy" misread.
- **Module:** Chat Wizard
- **Sub-Module:** Report
- **Section Hierarchy:** Chat Wizard > Report
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A completed execution (ideally with some not-run and healed tests).
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Click "Generate Report".	The Report step shows a pass-rate bar color-coded by threshold (≥ 90 / ≥ 70 / else).
2	Observe the metrics grid.	Total, Passed, Failed, Auto-Healed, Execution Time, and Healing Needed values are displayed.
3	Observe the not-run callout.	If any tests could not run, an amber callout lists them so the pass rate is not misread.

TC-048: Verify Push to GitHub resolves the connected repo and pushes scripts

- **Title Description:** Confirms the quick "Push to GitHub" action uses the connected repository config to publish scripts.
- **Module:** Chat Wizard
- **Sub-Module:** Git Publish
- **Section Hierarchy:** Chat Wizard > Git Publish
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** A GitHub repository is connected under Code Repositories.
- **Test Data:** repo_connected = true (GitHub)
- **References:** —

Step #	Step Description	Expected Result
1	On Execution Results or Report, click "Push to GitHub".	The connected GitHub repo is resolved from config.
2	Confirm the push.	Scripts are committed to the configured branch/scripts path.
3	Observe the outcome.	A success indication (and where applicable a PR/commit reference) is shown.

TC-049: Verify Publish to Repo assembles the client-deliverable bundle with PR/MR

- **Title Description:** Confirms the full Publish flow bundles specs, page objects, exported cases, docs, and CI workflow, and opens a branch + PR/MR (GitLab/Bitbucket) or pushes to branch (GitHub).
- **Module:** Chat Wizard
- **Sub-Module:** Git Publish
- **Section Hierarchy:** Chat Wizard > Git Publish
- **Priority:** High
- **Type:** E2E
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** A git repo is connected; a completed run with scripts and a report.
- **Test Data:** repo_connected = true
- **References:** —

Step #	Step Description	Expected Result
1	On the Report step, click "Publish to Repo..."	The Publish step shows the connected repo(s) with target branch and scripts folder.
2	Review the files-to-publish summary.	Specs, page objects, test cases (MD + CSV), and the report are listed.
3	Click "Publish".	A branch is created and files committed; for GitLab/Bitbucket a PR/MR is opened and its URL is displayed.

TC-050: Verify Publish empty state when no repository is connected

- **Title Description:** Confirms the Publish step guides the user to Code Repositories when no git integration exists.
- **Module:** Chat Wizard
- **Sub-Module:** Git Publish
- **Section Hierarchy:** Chat Wizard > Git Publish
- **Priority:** Medium
- **Type:** Negative
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** No git repository connected.
- **Test Data:** repo_connected = false (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Open the Publish step with no repo connected.	An empty state points the user to System Configuration → Code Repositories.
2	Attempt to publish.	No publish occurs; the guidance remains.

TC-051: Verify Report to Support dispatches to connected channels only when failures exist

- **Title Description:** Confirms the escalation action sends a failure/flaky summary to connected notification channels and is only offered when failing/flaky tests exist.
- **Module:** Chat Wizard
- **Sub-Module:** Support Escalation
- **Section Hierarchy:** Chat Wizard > Support Escalation
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** At least one notification channel (email/Slack/Teams) is connected.
- **Test Data:** notif_channel = email (connected)
- **References:** —

Step #	Step Description	Expected Result
1	Complete a fully-passing run.	The "Report to Support" action is not offered.
2	Complete a run with failures/flaky tests.	The "Report to Support" action is available.
3	Trigger it.	A failure/flaky summary is sent to all connected channels; a confirmation is shown.

TC-052: Verify the git-publish guard refuses to push into the framework monorepo

- **Title Description:** Confirms the FRAMEWORK_REPO_GUARD prevents publishing generated tests into the IntelliQE framework repository itself.
- **Module:** Chat Wizard
- **Sub-Module:** Git Publish
- **Section Hierarchy:** Chat Wizard > Git Publish
- **Priority:** High
- **Type:** Security
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** A repo config pointing at the framework monorepo URL.
- **Test Data:** repo_url = (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Configure a git repo whose URL matches the framework monorepo and attempt to publish.	The publish is refused with a clear guard error.
2	Observe the repository.	No commit/branch is created in the framework repo.

7. Module F — Generated Test Cases

TC-053: Verify the Generated Test Cases list filters by module, submodule, and search

- **Title Description:** Confirms saved runs can be filtered by module/submodule and searched by Jira ID, title, or user.
- **Module:** Generated Test Cases
- **Sub-Module:** List View
- **Section Hierarchy:** Generated Test Cases > List View
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** Multiple saved runs exist across modules.
- **Test Data:** module = Authentication search_term = QA-101
- **References:** —

Step #	Step Description	Expected Result
1	Open <code>/generated-tests</code> .	The list of saved runs is displayed with Jira ID, story, source, user, case count, review status, and created date.
2	Select a Module filter.	The Submodule filter enables and the list narrows to that module.
3	Enter a search term.	The list filters to matching runs; "Clear" resets the filters.

TC-054: Verify the run detail shows Test Cases, Test Data, and Reports tabs

- **Title Description:** Confirms opening a run reveals three tabs with the expected structured content.
- **Module:** Generated Test Cases
- **Sub-Module:** Detail View
- **Section Hierarchy:** Generated Test Cases > Detail View
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A saved run with test data and an execution report exists.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Click a run in the list.	The detail view opens with the story key and title.
2	Open the "Test Cases" tab.	Cases are listed with TC#, scenario (expandable to preconditions/steps/expected), and priority/type badges.
3	Open the "Test Data" tab.	Datasets, Field Data, Mappings, and Validations subsections show with counts (or a "No test data found" empty state).
4	Open the "Reports & Export" tab.	Execution summary, type/priority breakdowns, data coverage, and accessibility (WCAG) report render (or an empty state).

TC-055: Verify inline review toggle updates reviewed counts

- **Title Description:** Confirms marking a case reviewed updates the per-run reviewed/total progress and footer counts.
- **Module:** Generated Test Cases
- **Sub-Module:** Review Workflow
- **Section Hierarchy:** Generated Test Cases > Review Workflow
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A run with unreviewed cases.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Toggle a case from Pending to Reviewed.	The row shows "Reviewed" and the footer Reviewed count increments.
2	Select multiple cases and use the bulk "Mark as Reviewed".	All selected cases flip to Reviewed and the progress bar advances.
3	Return to the list view.	The run's review-status progress reflects the new counts.

TC-056: Verify adding a test case via the detail form

- **Title Description:** Confirms a new case can be added to an existing run with steps, priority, type, and precondition.
- **Module:** Generated Test Cases
- **Sub-Module:** Detail View
- **Section Hierarchy:** Generated Test Cases > Detail View
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A run detail is open.
- **Test Data:** title = "Verify session times out after inactivity" (created by Tester) priority = P2 type = negative
- **References:** —

Step #	Step Description	Expected Result
1	Click "Add Test Case".	An add form with title, priority, type, feature, dynamic steps, expected, and precondition is displayed.
2	Fill the fields with dynamic steps and submit.	The new case is added to the run and appears in the Test Cases table.
3	Observe the case count.	The run's case count increments accordingly.

TC-057: Verify tag filter chips refetch matching cases

- **Title Description:** Confirms selecting a tag chip (e.g. NEGATIVE, SMOKE) filters the case list server-side.
- **Module:** Generated Test Cases
- **Sub-Module:** Detail View
- **Section Hierarchy:** Generated Test Cases > Detail View
- **Priority:** Low
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A run with tagged cases.
- **Test Data:** tag = NEGATIVE
- **References:** —

Step #	Step Description	Expected Result
1	Click the "NEGATIVE" tag chip.	The case list refetches and shows only negative-tagged cases.
2	Clear the tag filter.	The full case list is restored.

TC-058: Verify CSV export from the list and Excel/Jira/TestRail export from detail

- **Title Description:** Confirms export options download files from both the list row action and the detail export dropdown.
- **Module:** Generated Test Cases
- **Sub-Module:** Export
- **Section Hierarchy:** Generated Test Cases > Export
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A saved run exists.
- **Test Data:** formats = Excel CSV, Jira-Xray, TestRail
- **References:** —

Step #	Step Description	Expected Result
1	From the list, click a run's "Export CSV" action.	A CSV file downloads for that run.
2	Open the run and use the Export dropdown (Excel CSV / Jira-Xray / TestRail).	Each option downloads a file in the selected format.

TC-059: Verify deleting a case and a whole run with confirmation

- **Title Description:** Confirms deletions require confirmation, snap the page back when emptied, and remove data.
- **Module:** Generated Test Cases
- **Sub-Module:** Deletion
- **Section Hierarchy:** Generated Test Cases > Deletion
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A run with multiple cases exists.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Delete a single case.	A confirmation modal appears; on confirm the case is removed.
2	Delete the last case on a page.	The view snaps back to a populated page.
3	Delete the whole run from the detail header.	A confirmation modal appears; on confirm the run is removed and the list no longer shows it.

TC-060: Verify Test Data tab surfaces DB-backed dataset references and validations

- **Title Description:** Confirms data-aware runs display dataset sources, `{{db:...}}` references, DB source config, and validation accept/reject reasons.
- **Module:** Generated Test Cases
- **Sub-Module:** Test Data
- **Section Hierarchy:** Generated Test Cases > Test Data
- **Priority:** Low
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** A run generated with data-aware datasets exists.
- **Test Data:** dataset_source = database (created by Tester)
- **References:** —

Step #	Step Description	Expected Result
1	Open the Test Data tab and expand a dataset.	Role, scenario, source badge (static/database/api/computed/mixed), and layer (ui/api) are shown.
2	Inspect a database-sourced dataset.	<code>{{db:...}}</code> references are highlighted and the DB source config (table/query) is shown.
3	Open the Validations subsection.	Accepted/rejected field values are listed with reasons.

8. Module G — Reports

TC-061: Verify the Reports list shows recent runs with color-coded pass rate

- **Title Description:** Confirms the executive Reports list presents up to the latest retained reports with pass-rate coloring and pagination.
- **Module:** Reports
- **Sub-Module:** Report List
- **Section Hierarchy:** Reports > Report List
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** At least one execution report exists.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Open <code>/reports</code> .	A list of recent reports shows title, module/origin, timestamp, results (passed/failed/total), and a color-coded pass rate.
2	With no reports, view the page.	An empty state prompts the user to run an execution from the Chat flow.
3	Change the page size.	Pagination reflows the list accordingly.

TC-062: Verify a report opens with Allure/Basic tabs and restores from storage

- **Title Description:** Confirms opening a report renders it in an iframe, offers Allure/Basic HTML tabs (each disabled if unavailable), and restores archived reports from Azure Storage.
- **Module:** Reports
- **Sub-Module:** Report Viewer
- **Section Hierarchy:** Reports > Report Viewer
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** A report exists (ideally previously archived to Azure Blob).
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Click a report row.	The full-page report opens; a toast indicates the report loaded (from Azure Storage where applicable).
2	Switch between the Allure and Basic HTML tabs.	The available variant renders in the iframe; an unavailable variant's tab is disabled.
3	Observe an unavailable report.	A "report isn't available" empty state is shown instead of a broken frame.

TC-063: Verify report export to Excel and PDF

- **Title Description:** Confirms the report viewer exports the report as XLSX and PDF.
- **Module:** Reports
- **Sub-Module:** Report Export
- **Section Hierarchy:** Reports > Report Export
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A report is open.
- **Test Data:** formats = xlsx, pdf
- **References:** —

Step #	Step Description	Expected Result
1	Click "Download Excel".	An <code>.xlsx</code> file downloads with the report data.
2	Click "Download PDF".	A <code>.pdf</code> file downloads rendering the report.

9. Module H — Bug Tracker

TC-064: Verify Bug Tracker lists bugs with type, severity, priority, and status

- **Title Description:** Confirms the Bug Tracker table renders auto- and manually-created bugs with all classification badges.
- **Module:** Bug Tracker
- **Sub-Module:** Bug List
- **Section Hierarchy:** Bug Tracker > Bug List
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** At least one failure/flaky/manual bug exists.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Open <code>/bug-tracker</code> .	Bugs are listed with Bug ID, Title, Severity, Priority, Status, Type badge, Module, and Updated.
2	Observe bug types.	Types are shown as Failure, Flaky, or Manual.
3	Observe a bug already raised in both ADO and JIRA.	Its select cell shows a green check indicating it is already tracked externally.

TC-065: Verify Bug Tracker filters and debounced search

- **Title Description:** Confirms filtering by status, severity, priority, and type plus a debounced search narrows the list.
- **Module:** Bug Tracker
- **Sub-Module:** Filters
- **Section Hierarchy:** Bug Tracker > Filters
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** Multiple bugs of varied classifications exist.
- **Test Data:** status = Open severity = critical type = Flaky search = login
- **References:** —

Step #	Step Description	Expected Result
1	Apply the Status, Severity, and Type filters.	The list narrows to bugs matching all selected filters.
2	Type into Search.	After the debounce, the list filters by title/module/assignee.
3	Click "Clear".	All filters reset and the full list returns.

TC-066: Verify creating a manual bug

- **Title Description:** Confirms a QA user can file a manual bug with the required title and optional detail fields.
- **Module:** Bug Tracker
- **Sub-Module:** Create Bug
- **Section Hierarchy:** Bug Tracker > Create Bug
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** On the Bug Tracker page.
- **Test Data:** title = "Dashboard KPI widget fails to load" (created by Tester) severity = high priority = P1 module = Dashboard
- **References:** —

Step #	Step Description	Expected Result
1	Click "Report Bug".	A create form with title (required), description, severity, priority, status, environment, module, steps, expected/actual, assignee, test case ID, and tags is displayed.
2	Submit with an empty title.	Validation blocks submission and flags the required title.
3	Fill the required fields and submit.	The bug is created with type <code>manual</code> and appears at the top of the list.

TC-067: Verify editing a bug uses optimistic-concurrency protection

- **Title Description:** Confirms an edit sends only changed fields with a concurrency token and is rejected if the bug changed elsewhere.
- **Module:** Bug Tracker
- **Sub-Module:** Edit Bug
- **Section Hierarchy:** Bug Tracker > Edit Bug
- **Priority:** Medium
- **Type:** Edge
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** A bug open for edit; ability to mutate it from a second session.
- **Test Data:** concurrent_update = yes (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Open a bug for edit in session A.	The edit form loads the current values and an <code>expected_updated_at</code> token.
2	From session B, update the same bug.	The bug's updated timestamp changes server-side.
3	Save the edit from session A.	The update is rejected with a 409 optimistic-concurrency error and the user is prompted to refresh.

TC-068: Verify the detail modal shows the activity timeline and revoke/reopen actions

- **Title Description:** Confirms the bug detail modal renders full metadata, tracker links, an activity timeline, and revoke/reopen controls.
- **Module:** Bug Tracker
- **Sub-Module:** Bug Detail
- **Section Hierarchy:** Bug Tracker > Bug Detail
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** A bug with history exists.
- **Test Data:** `revoke_reason` = "Not reproducible in latest build" (created by Tester)
- **References:** —

Step #	Step Description	Expected Result
1	Open a bug's detail.	Description, steps, expected/actual, resolution notes, metadata sidebar, and tracker links are displayed.
2	Observe the activity timeline.	Events (<code>created/status_changed/updated/auto_created/rerun_passed</code> , etc.) are listed with field-change diffs.
3	Click "Revoke" and supply a reason.	The bug moves to Revoked with the reason recorded.
4	Reopen a resolved/closed/revoked bug.	The bug returns to an active status.

TC-069: Verify raising selected bugs to Azure DevOps and JIRA

- **Title Description:** Confirms bulk-raising selected bugs creates external work items/issues and links them back, with per-item success/failure toasts.
- **Module:** Bug Tracker
- **Sub-Module:** External Trackers
- **Section Hierarchy:** Bug Tracker > External Trackers
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** ADO and/or JIRA connected under Requirement Sources; selectable bugs exist.
- **Test Data:** `ado_connected` = `true` `jira_connected` = `true`
- **References:** —

Step #	Step Description	Expected Result
1	Select one or more bugs and click "Raise Bug(s)".	A destination popup offers Azure DevOps and JIRA.
2	Raise to JIRA.	A JIRA issue is created and linked; toasts report raised/failed/already-present counts.
3	Raise to Azure DevOps.	An ADO work item is created and linked; the bug's tracker links update.
4	Attempt to raise with the tracker disconnected.	The action reports that a connection is required (System Configuration → Requirement Sources).

TC-070: Verify deleting a linked bug also removes its external item and updates live

- **Title Description:** Confirms deleting a bug removes its ADO/JIRA counterpart (with partial-failure warnings) and the list updates live via SSE.
- **Module:** Bug Tracker
- **Sub-Module:** External Trackers
- **Section Hierarchy:** Bug Tracker > External Trackers
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** A bug already raised to an external tracker; a second session open on the Bug Tracker.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Delete a bug that is linked to JIRA/ADO.	The linked external issue/work item is also deleted; partial failures raise a warning.
2	Observe the second session.	The bug list refetches live (SSE) and the deleted bug disappears without a manual refresh.

10. Module I — System Configuration

TC-071: Verify System Configuration is admin/qa gated and shows all tabs

- **Title Description:** Confirms the settings surface presents its eight configuration tabs to authorized roles.
- **Module:** System Configuration
- **Sub-Module:** Navigation
- **Section Hierarchy:** System Configuration > Navigation
- **Priority:** High
- **Type:** Smoke
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** Logged in as admin.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Open <code>/system-configuration</code> .	The tab nav shows General Settings, Application Setup, Requirement Sources, Storage, Code Repositories, Notifications, LLM Configuration, and Voice Assistant.
2	Attempt to open the page as a <code>data_analyst</code> .	The page is not accessible (route/sidebar gating).

TC-072: Verify adding an application with roles encrypts credentials at rest

- **Title Description:** Confirms an application-under-test can be added with test user roles whose passwords are encrypted client-side and stored encrypted at rest.
- **Module:** System Configuration
- **Sub-Module:** Application Setup
- **Section Hierarchy:** System Configuration > Application Setup
- **Priority:** Critical
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** On the Application Setup tab.
- **Test Data:** app_name = OrangeHRM Demo (created by Tester) base_url = <https://opensource-demo.orangehrmlive.com> (created by Tester) role = Admin / username=Admin / password=
- **References:** —

Step #	Step Description	Expected Result
1	Click "Add Application" and enter name, base URL, and environment.	The form accepts the values.
2	Add a Test User Role with username and password (show/hide works).	The role row is added; the password field is masked.
3	Save the application and inspect the request payload.	Sensitive fields are transit-encrypted (__ENC__) before send; the API response masks secrets.
4	Reopen the saved app.	The application persists; the password is not returned in plaintext.

TC-073: Verify connecting Jira validates the credentials before saving

- **Title Description:** Confirms the Jira connect flow validates via the Jira API and only then stores an encrypted auth header.
- **Module:** System Configuration
- **Sub-Module:** Requirement Sources
- **Section Hierarchy:** System Configuration > Requirement Sources
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Valid Jira Cloud credentials available.
- **Test Data:** jira_url = <https://.atlassian.net> (created by Tester) email = api_token =
- **References:** —

Step #	Step Description	Expected Result
1	Open the JIRA card → Connect and enter URL, email, and API token.	The connect modal accepts the fields.
2	Submit with an invalid token.	Validation fails and the connection is not saved.
3	Submit with valid credentials.	The connection is validated (e.g. /myself), saved with an encrypted auth header, and the card shows "Connected".
4	Edit the connection.	Secret fields are blank (never prefilled with ciphertext) and must be re-entered to change.

TC-074: Verify LLM Configuration supports provider/auth choice, test, save, and default

- **Title Description:** Confirms an admin can configure Anthropic (API key or Claude Code), test the connection, select a default model with per-agent overrides, and set the provider as default.
- **Module:** System Configuration
- **Sub-Module:** LLM Configuration
- **Section Hierarchy:** System Configuration > LLM Configuration
- **Priority:** Critical
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Admin on the LLM Configuration tab; a valid Anthropic key.
- **Test Data:** provider = Anthropic Claude auth_method = API Key api_key = default_model = claude-opus-4-8
- **References:** —

Step #	Step Description	Expected Result
1	Select Anthropic Claude and API Key auth, then enter the key.	The key field is masked; endpoint and model fields are shown.
2	Click "Test Connection".	The connection validates and live models populate the Default Model list.
3	Set per-agent model overrides and reasoning effort, then Save.	The configuration saves; the status badge shows "Connected"; the key is masked in responses.
4	Click "Set as Default".	The provider becomes the tenant default and is mirrored to the tenant Anthropic key for the worker.

TC-075: Verify LLM test connection reports invalid credentials distinctly

- **Title Description:** Confirms an invalid LLM key surfaces an "Invalid Credentials/Connection Failed" status rather than a generic error.
- **Module:** System Configuration
- **Sub-Module:** LLM Configuration
- **Section Hierarchy:** System Configuration > LLM Configuration
- **Priority:** High
- **Type:** Negative
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Admin on the LLM Configuration tab.
- **Test Data:** api_key = sk-ant-invalid (simulated)
- **References:** —

Step #	Step Description	Expected Result
1	Enter an invalid API key and click "Test Connection".	The test fails and the status shows Invalid Credentials / Connection Failed.
2	Observe the model list.	No live models are populated and the config is not marked Connected.

TC-076: Verify connecting Azure Blob storage validates before saving

- **Title Description:** Confirms report-archive storage (Azure Blob via connection string or managed identity) is validated prior to persisting.
- **Module:** System Configuration
- **Sub-Module:** Storage
- **Section Hierarchy:** System Configuration > Storage
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Admin on the Storage tab; valid Azure Blob details.
- **Test Data:** container = intelliqe-artifacts (created by Tester)
- **References:** —

Step #	Step Description	Expected Result
1	Enter the connection string (or managed-identity URL), container, and prefix, then click "Test".	The connectivity check succeeds.
2	Save the storage config.	The configuration persists and is used to archive reports (latest-10 retention).
3	Enter invalid details and test.	The test fails and the config is not saved.

TC-077: Verify sending a test email from the Notifications tab

- **Title Description:** Confirms the SMTP notification integration can send a test email using the saved configuration.
- **Module:** System Configuration
- **Sub-Module:** Notifications
- **Section Hierarchy:** System Configuration > Notifications
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Valid SMTP details on the Notifications tab.
- **Test Data:** smtp_to = qa.notifications@example.com (created by Tester)
- **References:** —

Step #	Step Description	Expected Result
1	Enter SMTP host/port/credentials and the To/Cc addresses.	The Outlook/SMTP fields accept the values.
2	Click the integration's "Test".	A test email is dispatched and a success result is shown inline.
3	Provide invalid SMTP details and test.	The test reports failure without saving broken credentials silently.

TC-078: Verify Voice Assistant (Tessa) settings and test playback

- **Title Description:** Confirms Tessa TTS can be enabled/disabled, voice/rate/pitch adjusted, and a test phrase played, with settings stored per browser.
- **Module:** System Configuration
- **Sub-Module:** Voice Assistant
- **Section Hierarchy:** System Configuration > Voice Assistant
- **Priority:** Low
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Browser with Web Speech voices available.
- **Test Data:** speech_rate = 1.2
- **References:** —

Step #	Step Description	Expected Result
1	Toggle the voice assistant on and choose a voice/language.	The selection is accepted.
2	Adjust Speech Rate and Pitch, then click "Test".	A spoken test phrase plays with the chosen settings.
3	Reload the page.	The voice settings persist for the browser.

11. Module J — User Management & Feature Toggles

TC-079: Verify User Management is admin-only and lists tenant users

- **Title Description:** Confirms non-admins are redirected and admins see the user table with role, status, and (for platform admins) tenant.
- **Module:** User Management
- **Sub-Module:** User List
- **Section Hierarchy:** User Management > User List
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** An admin and a non-admin user exist.
- **Test Data:** None
- **References:** —

Step #	Step Description	Expected Result
1	Open <code>/user-management</code> as a non-admin.	The guard redirects to <code>/chat</code> .
2	Open the page as an admin.	Users are listed with Username, Full Name, Role, Status, and Last Updated.
3	Observe the Tenant column.	It is shown only for platform admins (cross-tenant visibility).

TC-080: Verify creating, editing, and deactivating a user

- **Title Description:** Confirms an admin can create a user, edit them (username locked, blank password keeps current), and toggle active status, with self-actions blocked.
- **Module:** User Management
- **Sub-Module:** User CRUD
- **Section Hierarchy:** User Management > User CRUD
- **Priority:** High
- **Type:** Functional
- **Automation Type:** Automated
- **Template:** Test Case (Steps)
- **Preconditions:** Logged in as admin.
- **Test Data:** username = qa_new_user (created by Tester) role = QA Engineer password =
- **References:** —

Step #	Step Description	Expected Result
1	Click "New User", fill username/full name/email/role/password (matching), and save.	The user is created and appears in the list.
2	Edit the user; leave the password blank and change the role.	Username is locked; the blank password keeps the current one; the role updates.
3	Toggle the user's status to Inactive.	The status flips to Inactive.
4	Attempt to deactivate or delete your own account.	The action is blocked (self-deactivate/self-delete prevented).

TC-081: Verify Feature Toggles gate page availability per workspace

- **Title Description:** Confirms an admin can disable a feature and the corresponding page/nav becomes unavailable, without locking out the toggles page itself.
- **Module:** Feature Toggles
- **Sub-Module:** Toggle Management
- **Section Hierarchy:** Feature Toggles > Toggle Management
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Admin access to the (hidden) Feature Toggles page.
- **Test Data:** feature = reports
- **References:** —

Step #	Step Description	Expected Result
1	Disable the "Reports" feature and Save changes.	The change persists (explicit booleans written per feature×role).
2	Navigate to <code>/reports</code> .	A friendly "feature unavailable" panel is shown instead of the reports list.
3	Observe the Feature Toggles page itself.	It remains reachable (excluded from the catalog so admins can't lock themselves out).
4	Disable "chat" and open <code>/chat</code> .	The Layout shows a FeatureUnavailable panel rather than mounting the Chat wizard.

TC-082: Verify sidebar visibility respects role, hidden paths, and server menu-config

- **Title Description:** Confirms navigation items are gated by role, the `HIDDEN_PATHS` list, per-tenant `allowedPaths`, and feature toggles together.
- **Module:** User Management
- **Sub-Module:** Navigation Gating
- **Section Hierarchy:** User Management > Navigation Gating
- **Priority:** Medium
- **Type:** Functional
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** A tenant with a restricted `menu-config allowedPaths` set.
- **Test Data:** `allowedPaths` = `/chat, /generated-tests, /reports` (created by Tester)
- **References:** —

Step #	Step Description	Expected Result
1	Log in as a <code>qa_engineer</code> in the restricted tenant.	The sidebar shows only allowed, role-permitted, non-hidden, feature-enabled items.
2	Observe admin-only items as a non-admin.	User Management/System Configuration are hidden per role rules.
3	Confirm hidden paths (Agent Performance, Feature Toggles).	These are not shown in the sidebar regardless of role.

12. Module K — Cross-cutting: Multi-tenancy, Security & Pipeline Guards

TC-083: Verify tenant isolation prevents cross-tenant data and SSE access

- **Title Description:** Confirms a user in one tenant cannot read another tenant's runs, reports, bugs, or subscribe to another tenant's pipeline events.
- **Module:** Security
- **Sub-Module:** Multi-Tenancy
- **Section Hierarchy:** Security > Multi-Tenancy
- **Priority:** Critical
- **Type:** Security
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Two non-platform tenants each with their own data and a `runId`.
- **Test Data:** `tenant_A_runId` = (created by Tester) `tenant_B_user` =
- **References:** —

Step #	Step Description	Expected Result
1	As a tenant-B user, call <code>GET /api/test-cases/<tenant_A_runId></code> .	The response does not return tenant A's data (filtered by <code>tenant_id</code>).
2	As a tenant-B user, attempt to open the Reports/Bug entries of tenant A.	Tenant A's records are not visible.
3	Attempt to subscribe to <code>/api/pipeline-events/<tenant_A_runId></code> as tenant B.	The subscription is rejected (composite/cross-tenant keys refused).
4	Repeat step 1 as a platform-tenant admin.	Cross-tenant read is permitted for platform admins only.

TC-084: Verify pipeline convergence guards cap iteration, budget, and healing

- **Title Description:** Confirms the orchestrator terminates runaway runs via max-iterations/budget caps and the in-process healer stops on time-budget or no-progress, without fabricating results.
- **Module:** AI Pipeline
- **Sub-Module:** Convergence Guards
- **Section Hierarchy:** AI Pipeline > Convergence Guards
- **Priority:** High
- **Type:** Edge
- **Automation Type:** Manual
- **Template:** Test Case (Steps)
- **Preconditions:** Ability to run the DB-queued pipeline and force repeated stage failures.
- **Test Data:** maxIterations = configured limit budgetPerRunUsd = configured cap HEAL_MAX_PASSES = 2 (default)
- **References:** —

Step #	Step Description	Expected Result
1	Start a run and force a stage to keep re-queuing past <code>maxIterations</code> .	The orchestrator terminates the run in a <code>fixme</code> terminal state and emits a completion event.
2	Force per-run cost to exceed <code>budgetPerRunUsd</code> .	The run is cancelled with an over-budget status rather than continuing.
3	Drive healing on an unfixable failure past the pass/time budget.	Healing stops after the max passes / time budget / identical-signature detection and marks the run for human review.
4	Inspect the results throughout.	No stage reports a fabricated pass; unrunnable tests remain <code>not_run</code> with a reason.

13. Definition of Done

A test cycle against this plan is complete when:

- Every case above has a recorded result (Pass / Fail / Blocked) with evidence (screenshot, report link, or API response) attached.
- All **Critical** and **High** cases pass, or failures are logged as bugs in the Bug Tracker with severity and a linked run/test-case reference.
- No security case (TC-005, TC-052, TC-083) reveals plaintext secrets, cross-tenant leakage, or a bypass of the framework-repo guard.
- The plan imports cleanly into TestRail (Section 7 of the authoring standard) and TestLink (Section 8) with no manual field fixes.

14. Traceability — Modules ↔ Backend Surface

Plan Module	Primary Frontend	Primary Backend
A. Authentication	LoginPage , AuthContext	POST /api/auth/login , /signup , auth.middleware
B. Requirement Sources	ChatPage	/api/jira , /api/azure-devops , /api/confluence , /api/sharepoint , /api/document/extract
C. Generation & Review	ChatPage	POST /api/generate , generatorAgent , plannerAgent , requirementAgent , auditAgent
D. Scripts/Execution/Healing	ChatPage	pipeline-flow /scripts /execute /heal , scriptAgent , executionAgent , healingAgent
E. Reporting & Publish	ChatPage , ReportsPage	/api/reports , /api/allure , /api/git/publish , notifications
F. Generated Test Cases	GeneratedTestCasesPage	/api/test-cases , /api/artifacts
G. Reports	ReportsPage	/api/reports , /api/allure
H. Bug Tracker	BugTrackerPage	/api/bugs (+ ADO/JIRA push)
I. System Configuration	SystemConfigurationPage	/api/configurations , /api/llm-config , /api/storage , /api/tenant-settings

Plan Module	Primary Frontend	Primary Backend
J. Users & Toggles	UserManagementPage , FeatureTogglesPage	/api/users , /api/feature-toggles , menu-config
K. Cross-cutting	(all)	auth.middleware , orchestrator , worker, utils/crypto